

DATABASE PROGRAMMING (BIIT 1031)

SEMESTER 2 2025/2026

FINAL REPORT

LECTURER NAME:

DR. NURHAFIZAH BINTI MAHRI

SECTION:

4

GROUP MEMBERS:

NO	NAME	MATRIC NO
1	NURUL HANA BINTI ABDUL HALIM	2315932
2	CHE NURUL LIYANA BINTI CHE MAT NOOR	2317600
3	DAYANA BATRISYA BINTI RAMILE	2318872
4	NUR NAJIHAH BINTI MOHD ZAMRISHAM	2319880

TASK 1

1.1 SCENARIO

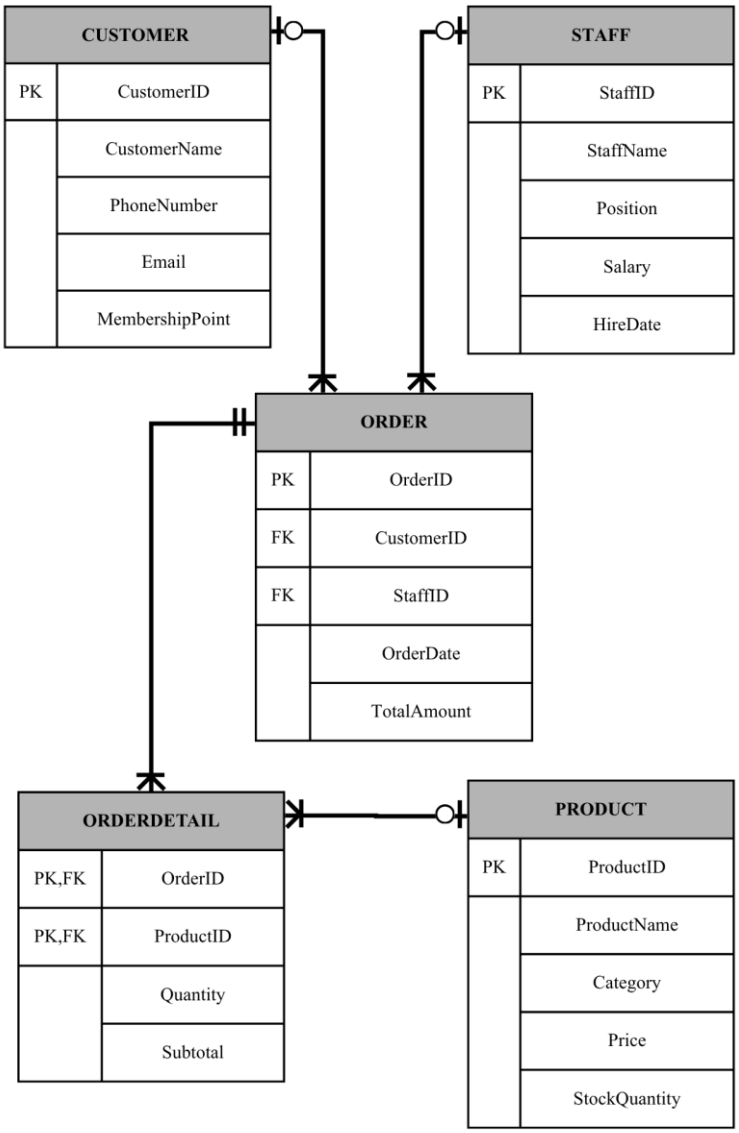
Ballerina Cappucina is a newly opened local coffee shop that serves a wide selection of drinks and desserts. Currently, the management team faces challenges in monitoring product stock levels, tracking daily sales transactions, and preparing accurate sales reports as most of the business operations are handled manually. Therefore, the team is planning to introduce a database system to organize information more efficiently, especially for employees, customers, products, and customer purchases. The database should store employee information, such as staff position and salary to help monitor employee records and sales handled by staff members. Customers who visit the coffee shop can order different types of drinks and food items, including various types of coffee, tea, and cakes. Each order is processed by a staff member working at the cashier counter. A customer can make many orders over time, but each order is linked to only one customer. The coffee shop offers a variety of products, and each product contains details such as product category, selling price, and available stock quantity. The database stores order item information in a separate section as an order may include several products, and the same product can appear in different orders. This allows the shop to record the quantity purchased and subtotal for every product in an order. By using this database system, Ballerina Cappucina will be able to maintain customer information, monitor product stock, record customer purchases, supervise staff activities, and prepare sales reports more effectively.

1.2 BUSINESS RULE

1. One customer can place many orders.
2. Each order belongs to one customer only.
3. One staff member handles many orders.
4. Each order is handled by one staff member.
5. One order can contain many products.
6. One product can appear in many orders.
7. OrderDetail record quantity and subtotal for each product in an order.
8. Stock quantity in product cannot be less than zero.
9. Each product must have a price greater than zero.

TASK 2

2.1 ENTITIES RELATIONSHIP DIAGRAM (ERD)



TASK 3

3.1 TABLES DDL SCRIPT

1. **Create table Staff**

```
CREATE TABLE Staff (
    StaffID INT PRIMARY KEY,
    StaffName VARCHAR2(100) NOT NULL,
    Position VARCHAR2(50),
    Salary NUMBER(10,2) NOT NULL CHECK (Salary > 0),
    HireDate DATE NOT NULL );
```

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	STAFFID	NUMBER(38,0)	No	(null)	1	(null)
2	STAFFNAME	VARCHAR2(100 BYTE)	No	(null)	2	(null)
3	POSITION	VARCHAR2(50 BYTE)	Yes	(null)	3	(null)
4	SALARY	NUMBER(10,2)	No	(null)	4	(null)
5	HIREDATE	DATE	No	(null)	5	(null)

2. **Create table Customers**

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR2(100) NOT NULL,
    PhoneNumber VARCHAR2(20),
    Email VARCHAR2(100),
    MembershipPoint INT );
```

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	CUSTOMERID	NUMBER(38,0)	No	(null)	1	(null)
2	CUSTOMERNAME	VARCHAR2(100 BYTE)	No	(null)	2	(null)
3	PHONENUMBER	VARCHAR2(20 BYTE)	Yes	(null)	3	(null)
4	EMAIL	VARCHAR2(100 BYTE)	Yes	(null)	4	(null)
5	MEMBERSHIPPOINT	NUMBER(38,0)	Yes	(null)	5	(null)

3. **Create table Orders**

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    OrderDate DATE NOT NULL,
    TotalAmount NUMBER(10,2) NOT NULL CHECK (TotalAmount > 0),
    CustomerID INT NOT NULL,
    StaffID INT NOT NULL,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (StaffID) REFERENCES Staff(StaffID) );
```

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ORDERID	NUMBER(38,0)	No	(null)	1	(null)
2	ORDERDATE	DATE	No	(null)	2	(null)
3	TOTALAMOUNT	NUMBER(10,2)	No	(null)	3	(null)
4	CUSTOMERID	NUMBER(38,0)	No	(null)	4	(null)
5	STAFFID	NUMBER(38,0)	No	(null)	5	(null)

4. **Create table Product**

```
CREATE TABLE Product (
   ProductID INT PRIMARY KEY,
   ProductName VARCHAR2(100) NOT NULL,
   Category VARCHAR2(20) NOT NULL,
   Price NUMBER(10,2) NOT NULL CHECK (Price > 0),
   StockQuantity INT NOT NULL CHECK (StockQuantity >= 0) );
```

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	PRODUCTID	NUMBER(38,0)	No	(null)	1	(null)
2	PRODUCTNAME	VARCHAR2(100 BYTE)	No	(null)	2	(null)
3	CATEGORY	VARCHAR2(20 BYTE)	No	(null)	3	(null)
4	PRICE	NUMBER(10,2)	No	(null)	4	(null)
5	STOCKQUANTITY	NUMBER(38,0)	No	(null)	5	(null)

5. **Create table OrderDetails**

```
CREATE TABLE OrderDetail (
   OrderID INT NOT NULL,
   ProductID INT NOT NULL,
   Subtotal NUMBER(10,2)NOT NULL CHECK (Subtotal > 0),
   Quantity INT NOT NULL CHECK (Quantity > 0),
   PRIMARY KEY (OrderID, ProductID),
   FOREIGN KEY (OrderID) REFERENCES Orders(OrderID),
   FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
);
```

	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	ORDERID	NUMBER(38,0)	No	(null)	1	(null)
2	PRODUCTID	NUMBER(38,0)	No	(null)	2	(null)
3	SUBTOTAL	NUMBER(10,2)	No	(null)	3	(null)
4	QUANTITY	NUMBER(38,0)	No	(null)	4	(null)

3.2 DATA DICTIONARY

Table 1: CUSTOMER

Description: Store customer information including contact number, email address and membership point for customers who purchase any products.

ATTRIBUTE	DATA TYPE	DESCRIPTION
CustomerID	INT	Unique ID for customer
CustomerName	VARCHAR2(100)	Customer full name
PhoneNumber	VARCHAR2(20)	Customer contact number
Email	VARCHAR2(100)	Customer email address
MembershipPoint	INT	Customer reward

Table 2: STAFF

Description: Store employee detailed information including job position, salary and date their start working.

ATTRIBUTE	DATA TYPE	DESCRIPTION
StaffID	INT	Staff ID for staff
StaffName	VARCHAR2(100)	Staff full name
Position	VARCHAR2(50)	Job position
Salary	NUMBER(10,2)	Staff salary
HireDate	DATE	Staff hiring date

Table 3: ORDER

Description: Store customer orders handle by staff members.

ATTRIBUTE	DATA TYPE	DESCRIPTION
OrderID	INT	Unique order ID
OrderDate	DATE	Date and time of order
TotalAmount	NUMBER(10,2)	Total order amount
CustomerID	INT	References Customer table
StaffID	INT	References Staff tables

Table 4: ORDER DETAIL

Description: Store detailed information about products in each order transaction.

ATTRIBUTE	DATA TYPE	DESCRIPTION
OrderID	INT	References Order table
ProductID	INT	References Product table
Quantity	INT	Quantity ordered
Subtotal	NUMBER(10,2)	Product subtotal amount

Table 5: PRODUCT

Description: Store information about products sold including category, price and stock quantity.

ATTRIBUTE	DATA TYPE	DESCRIPTION
ProductID	INT	Unique product ID
ProductName	VARCHAR2(100)	Product name
Category	VARCHAR2(50)	Product category
Price	NUMBER(10,2)	Product selling price
StockQuantity	INT	Available stock quantity

TASK 4

4.1 POPULATE TABLES USING DML

1. **Populate table Staff**

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (1, 'Dexter Morgan', 'Manager', 5500.00, TO_DATE('2023-01-15', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (2, 'Debra Morgan', 'Barista', 3200.00, TO_DATE('2023-02-10', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (3, 'James Doakes', 'Barista', 3100.00, TO_DATE('2023-03-05', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (4, 'Angel Batista', 'Cashier', 3000.00, TO_DATE('2023-04-12', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (5, 'Vince Masuka', 'Barista', 3200.00, TO_DATE('2023-05-20', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (6, 'Maria LaGuerta', 'Cleaner', 2500.00, TO_DATE('2023-06-01', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (7, 'Joey Quinn', 'Barista', 3150.00, TO_DATE('2023-07-15', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (8, 'Arthur Mitchell', 'Shift Lead', 4000.00, TO_DATE('2023-08-22', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (9, 'Hannah McKay', 'Cashier', 3000.00, TO_DATE('2023-09-10', 'YYYY-MM-DD'));

INSERT INTO Staff (StaffID, StaffName, Position, Salary, HireDate) VALUES (10, 'Harry Morgan', 'Barista', 3200.00, TO_DATE('2023-10-05', 'YYYY-MM-DD'));

	STAFFID	STAFFNAME	POSITION	SALARY	HIREDATE
1	1	Dexter Morgan	Manager	5500	15/01/2023
2	2	Debra Morgan	Barista	3200	10/02/2023
3	3	James Doakes	Barista	3100	05/03/2023
4	4	Angel Batista	Cashier	3000	12/04/2023
5	5	Vince Masuka	Barista	3200	20/05/2023
6	6	Maria LaGuerta	Cleaner	2500	01/06/2023
7	7	Joey Quinn	Barista	3150	15/07/2023
8	8	Arthur Mitchell	Shift Lead	4000	22/08/2023
9	9	Hannah McKay	Cashier	3000	10/09/2023
10	10	Harry Morgan	Barista	3200	05/10/2023

2. **Populate table Customer**

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (1, 'Walter White', '012-9428100', 'walter@example.com', 150);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (2, 'Jesse Pinkman', '013-5550102', 'jesse@example.com', 240);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (3, 'Skyler White', '014-5550103', 'skyler@example.com', 50);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (4, 'Hank Schrader', '016-5550104', 'hank@example.com', 310);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (5, 'Saul Goodman', '017-5550105', 'saul@example.com', 500);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (6, 'Gustavo Fring', '018-5550106', 'gus@example.com', 420);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (7, 'Mike Ehrmantraut', '019-5550107', 'mike.e@example.com', 10);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (8, 'Marie Schrader', '011-5550108', 'marie@example.com', 85);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (9, 'Tuco Salamanca', '012-5550109', 'tuco@example.com', 999);

INSERT INTO Customer (CustomerID, CustomerName, PhoneNumber, Email, MembershipPoint) VALUES (10, 'Gale Boetticher', '013-5550110', 'gale@example.com', 120);

	CUSTOMERID	CUSTOMERNAME	PHONENUMBER	EMAIL	MEMBERSHIPPOINT
1	1	Walter White	012-9428100	walter@example.com	150
2	2	Jesse Pinkman	013-5550102	jesse@example.com	240
3	3	Skyler White	014-5550103	skyler@example.com	50
4	4	Hank Schrader	016-5550104	hank@example.com	310
5	5	Saul Goodman	017-5550105	saul@example.com	500
6	6	Gustavo Fring	018-5550106	gus@example.com	420
7	7	Mike Ehrmantraut	019-5550107	mike.e@example.com	10
8	8	Marie Schrader	011-5550108	marie@example.com	85
9	9	Tuco Salamanca	012-5550109	tuco@example.com	999
10	10	Gale Boetticher	013-5550110	gale@example.com	120

3. Populate table Orders

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (101, TO_DATE('2024-05-01', 'YYYY-MM-DD'), 8.00, 1, 4);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (102, TO_DATE('2024-05-01', 'YYYY-MM-DD'), 4.50, 2, 2);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (103, TO_DATE('2024-05-01', 'YYYY-MM-DD'), 12.75, 5, 4);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (104, TO_DATE('2024-05-02', 'YYYY-MM-DD'), 3.50, 3, 9);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (105, TO_DATE('2024-05-02', 'YYYY-MM-DD'), 15.50, 6, 2);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (106, TO_DATE('2024-05-03', 'YYYY-MM-DD'), 5.50, 7, 3);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (107, TO_DATE('2024-05-03', 'YYYY-MM-DD'), 9.00, 4, 3);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (108, TO_DATE('2024-05-04', 'YYYY-MM-DD'), 3.00, 8, 4);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (109, TO_DATE('2024-05-04', 'YYYY-MM-DD'), 7.50, 9, 2);

INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID, StaffID) VALUES (110, TO_DATE('2024-05-04', 'YYYY-MM-DD'), 4.25, 10, 9);

	ORDERID	ORDERDATE	TOTALAMOUNT	CUSTOMERID	STAFFID
1	101	01/05/2024	8	1	4
2	102	01/05/2024	4.5	2	2
3	103	01/05/2024	12.75	5	4
4	104	02/05/2024	3.5	3	9
5	105	02/05/2024	15.5	6	2
6	106	03/05/2024	5.5	7	3
7	107	03/05/2024	9	4	3
8	108	04/05/2024	3	8	4
9	109	04/05/2024	7.5	9	2
10	110	04/05/2024	4.25	10	9

4. **Populate table Product**

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (1, 'Espresso', 'Drink', 3.50, 100);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (2, 'Latte', 'Drink', 4.50, 80);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (3, 'Cappuccino', 'Drink', 4.25, 80);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (4, 'Americano', 'Drink', 3.00, 150);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (5, 'Mocha', 'Drink', 5.00, 60);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (6, 'Croissant', 'Food', 2.50, 40);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (7, 'Blueberry Muffin', 'Food', 3.00, 30);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (8, 'Cheesecake', 'Food', 5.50, 20);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (9, 'Iced Tea', 'Drink', 3.25, 100);

INSERT INTO Product (ProductID, ProductName, Category, Price, StockQuantity) VALUES (10, 'Hot Chocolate', 'Drink', 4.00, 50);

	PRODUCTID	PRODUCTNAME	CATEGORY	PRICE	STOCKQUANTITY
1	1	Espresso	Drink	3.5	100
2	2	Latte	Drink	4.5	80
3	3	Cappuccino	Drink	4.25	80
4	4	Americano	Drink	3	150
5	5	Mocha	Drink	5	60
6	6	Croissant	Food	2.5	40
7	7	Blueberry Muffin	Food	3	30
8	8	Cheesecake	Food	5.5	20
9	9	Iced Tea	Drink	3.25	100
10	10	Hot Chocolate	Drink	4	50

5. **Populate table OrderDetail**

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (101, 1, 1, 3.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (101, 2, 1, 4.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (102, 2, 1, 4.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (103, 3, 3, 12.75);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (104, 1, 1, 3.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (105, 8, 2, 11.00);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (105, 2, 1, 4.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (106, 8, 1, 5.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (107, 10, 1, 4.00);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (107, 5, 1, 5.00);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (108, 4, 1, 3.00);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (109, 6, 3, 7.50);

INSERT INTO OrderDetail (OrderID, ProductID, Quantity, Subtotal) VALUES (110, 3, 1, 4.25);

	ORDERID	PRODUCTID	SUBTOTAL	QUANTITY
1	101	1	3.5	1
2	101	2	4.5	1
3	102	2	4.5	1
4	103	3	12.75	3
5	104	1	3.5	1
6	105	8	11	2
7	105	2	4.5	1
8	106	8	5.5	1
9	107	10	4	1
10	107	5	5	1
11	108	4	3	1
12	109	6	7.5	3
13	110	3	4.25	1

TASK 5

5.1 DEFINE CONSTRAINTS

1. Define Primary Key

An attribute that uniquely identifies each record in a table. It cannot contain any duplicate or have a NULL value.

TABLE	PRIMARY KEY (PK)
CUSTOMER	CustomerID
STAFF	staffID
ORDERS	ProductID
PRODUCT	OrderID
ORDER DETAIL	OrderID
	ProductID

2. Define Foreign Key

An attribute that connects one table to another table by refer to primary key of another table. It maintains referential integrity between tables.

TABLE	FOREIGN KEY (FK)	REFERENCES
CUSTOMER	-	-
STAFF	-	-
PRODUCT	-	-
ORDERS	CustomerID	CUSTOMER(CustomerID)
	StaffID	STAFF(StaffID)
ORDERDETAIL	OrderID	ORDER(OrderID)
	ProductID	PRODUCT(ProductID)

3. Define Check

Restrict value inserted into attributes based on specific condition.

TABLE	CHECK CONSTRAINT
CUSTOMER	-
STAFF	CHECK (Salary > 0)
ORDERS	CHECK (TotalAmount > 0)
PRODUCT	CHECK (Price > 0)
	CHECK (StockQuantity >= 0)
ORDER DETAIL	CHECK (Quantity > 0),
	CHECK (Subtotal > 0)

4. Define Not Null

Ensures that a column cannot store NULL or empty values. Data must always be provided for the column.

TABLE	Not Null
CUSTOMER	CustomerName
STAFF	StaffName
	Salary
	HireDate
ORDERS	OrderDate
	CustomerID
	StaffID
	TotalAmount
PRODUCT	ProductName
	Price
	Category
	StockQuantity
ORDER DETAIL	OrderID
	ProductID
	Subtotal
	Quantity

TASK 6

Write (5) SQL statements to query, add, modify, and/or remove data that are useful to the scenario (e.g. add suppliers, add change salary, update inventory, etc

1- Identify Low Stock Products

```
SELECT ProductID, ProductName, StockQuantity
FROM Product
WHERE StockQuantity <= 50;
```

OUTPUT:

PRODUCTID	PRODUCTNAME	STOCKQUANTITY
6	Croissant	40
7	Blueberry Muffin	30
8	Cheesecake	20
10	Hot Chocolate	50

2- Give 10% Raise to Three Staffs With Most Processed Orders

```
UPDATE Staff
SET Salary = Salary * 1.10
WHERE StaffID IN (
   SELECT StaffID
   FROM (
      SELECT StaffID
      FROM Orders
      GROUP BY StaffID
      ORDER BY COUNT(OrderID) DESC   )
   WHERE ROWNUM <= 3);
```

BEFORE UPDATE:

STAFFID	STAFFNAME	POSITION	SALARY	HIREDATE
1	Dexter Morgan	Manager	5500	15/01/2023
2	Debra Morgan	Barista	3200	10/02/2023
3	James Doakes	Barista	3100	05/03/2023
4	Angel Batista	Cashier	3000	12/04/2023
5	Vince Masuka	Barista	3200	20/05/2023
6	Maria LaGuerta	Cleaner	2500	01/06/2023
7	Joey Quinn	Barista	3150	15/07/2023
8	Arthur Mitchell	Shift Lead	4000	22/08/2023
9	Hannah McKay	Cashier	3000	10/09/2023
10	Harry Morgan	Barista	3200	05/10/2023

AFTER UPDATE:

STAFFID	STAFFNAME	POSITION	SALARY	HIREDATE
1	Dexter Morgan	Manager	5500	15/01/2023
2	Debra Morgan	Barista	3520	10/02/2023
3	James Doakes	Barista	3410	05/03/2023
4	Angel Batista	Cashier	3300	12/04/2023
5	Vince Masuka	Barista	3200	20/05/2023
6	Maria LaGuerta	Cleaner	2500	01/06/2023
7	Joey Quinn	Barista	3150	15/07/2023
8	Arthur Mitchell	Shift Lead	4000	22/08/2023
9	Hannah McKay	Cashier	3000	10/09/2023
10	Harry Morgan	Barista	3200	05/10/2023

3- Display Top Three Products Ranked by Highest Revenue With Units Sold

```
SELECT * FROM (
SELECT p.ProductName AS "Product Name",
 SUM(d.Quantity) AS "Units Sold",
 SUM(d.Subtotal) AS "Revenue"
FROM Product p
JOIN OrderDetail d ON p.ProductID = d.ProductID
```

```
GROUP BY p.ProductName
ORDER BY "Revenue" DESC)
WHERE ROWNUM <= 3;
```

OUTPUT:

Product Name	Units Sold	Revenue
Cappuccino	4	17
Cheesecake	3	16.5
Latte	3	13.5

4- Award 10 Bonus Membership Points to Three Top Spending Customers

```
UPDATE Customer
SET MembershipPoint = MembershipPoint + 10
WHERE CustomerID IN (
 SELECT CustomerID FROM (
 SELECT CustomerID
 FROM Orders
 GROUP BY CustomerID
 ORDER BY SUM(TotalAmount) DESC)
 WHERE ROWNUM <= 3) ;
```

BEFORE UPDATE:

CUSTOMERID	CUSTOMERNAME	PHONENUMBER	EMAIL	MEMBERS...
1	Walter White	012-9428100	walter@example.com	150
2	Jesse Pinkman	013-5550102	jesse@example.com	240
3	Skyler White	014-5550103	skyler@example.com	50
4	Hank Schrader	016-5550104	hank@example.com	310
5	Saul Goodman	017-5550105	saul@example.com	500
6	Gustavo Fring	018-5550106	gus@example.com	420
7	Mike Ehrmantraut	019-5550107	mike.e@example.com	10
8	Marie Schrader	011-5550108	marie@example.com	85
9	Tuco Salamanca	012-5550109	tuco@example.com	999
10	Gale Boetticher	013-5550110	gale@example.com	120

AFTER UPDATE:

CUSTOMERID	CUSTOMERNAME	PHONENUMBER	EMAIL	MEMBERSHIPPOINT
1	Walter White	012-9428100	walter@example.com	150
2	Jesse Pinkman	013-5550102	jesse@example.com	240
3	Skyler White	014-5550103	skyler@example.com	50
4	Hank Schrader	016-5550104	hank@example.com	320
5	Saul Goodman	017-5550105	saul@example.com	510
6	Gustavo Fring	018-5550106	gus@example.com	430
7	Mike Ehrmantraut	019-5550107	mike.e@example.com	10
8	Marie Schrader	011-5550108	marie@example.com	85
9	Tuco Salamanca	012-5550109	tuco@example.com	999
10	Gale Boetticher	013-5550110	gale@example.com	120

5- Compare Total Units Sold Between Food and Drink Categories

```
SELECT p.Category, SUM(d.Quantity) AS "Units Sold"
FROM Product p
JOIN OrderDetail d ON p.ProductID = d.ProductID
GROUP BY p.Category
ORDER BY SUM(d.Quantity) DESC;
```

OUTPUT:

CATEGORY	Units Sold
Drink	12
Food	6

TASK 7

PL/SQL Scripts:

I. Write two (2) procedures

Procedure 1: Adding 10 points to membership points based on the customer id entered by the user.

```
CREATE OR REPLACE PROCEDURE add_membership_point (
   p_customer_id IN customer.customerid%TYPE
) IS
BEGIN
   UPDATE customer
   SET membershippoint = membershippoint + 10
   WHERE customerid = p_customer_id;
   DBMS_OUTPUT.PUT_LINE('Successfully added 10 loyalty points to Customer ID: ' ||
p_customer_id);
END;
/
```

Anonymous Block to call Procedure 1:

```
SET SERVEROUTPUT ON;
SET VERIFY OFF;
ACCEPT customer_id PROMPT 'Enter Customer ID : '
DECLARE
   v_customer_id customer.customerid%TYPE := &customer_id;
BEGIN
   add_membership_point(p_customer_id => v_customer_id);
END;
/
```

Example: Add membership point for customer ID = 7.

```
Successfully added 10 loyalty points to Customer ID: 7


PL/SQL procedure successfully completed.
```

Before:

7	7	Mike Ehrmantraut	019-5550107	mike.e@example.com	10
8	8	Marie Schrader	011-5550108	marie@example.com	85
9	9	Tuco Salamanca	012-5550109	tuco@example.com	999

After:

7	7	Mike Ehrmantraut	019-5550107	mike.e@example.com	20
8	8	Marie Schrader	011-5550108	marie@example.com	85
9	9	Tuco Salamanca	012-5550109	tuco@example.com	999

Procedure 2: Deduct the stock quantity for selected product id by asking quantity used.

```
CREATE OR REPLACE PROCEDURE deduct_stock (
   p_product_id IN product.productid%TYPE,
   p_quantity_used IN NUMBER
) IS
BEGIN
   UPDATE product
   SET stockquantity = stockquantity - p_quantity_used
   WHERE productid = p_product_id;
   DBMS_OUTPUT.PUT_LINE('Successfully deducted Product Stock for Product: ' ||
p_product_id);
END;
/
```

Anonymous Block for Procedure 2:

```
SET SERVEROUTPUT ON;
SET VERIFY OFF;
ACCEPT prod_id PROMPT 'Enter Product ID : '
ACCEPT prod_quantity PROMPT 'Enter Quantity Used : '
DECLARE
   v_prod_id product.productid%TYPE := &prod_id;
   v_prod_quantity NUMBER := &prod_quantity;
BEGIN
   deduct_stock(p_product_id => v_prod_id, p_quantity_used => v_prod_quantity);
END;
/
```

Example: Deduct stock for product ID = 10 by 3.

```
Procedure DEDUCT_STOCK compiled

Successfully deducted Product Stock for Product: 10


PL/SQL procedure successfully completed.
```

Before:

10	10	Hot Chocolate	Drink	4	50

After:

10	10	Hot Chocolate	Drink	4	47

TASK 8

PL/SQL Scripts:

II. and two (2) functions that are useful to the scenario. Call the functions from an anonymous block.

FUNCTION 1: Get final amount by adding 6% tax for each order id inserted.

```
CREATE OR REPLACE FUNCTION get_final_total_by_order (p_order_id IN INT)
RETURN NUMBER
IS
   v_total_amount NUMBER(10,2);
   v_tax NUMBER(10,2):=0.06;
   v_final_amount NUMBER(10,2);
BEGIN
   DBMS_OUTPUT.PUT_LINE('--- Function: Getting Final Total Including Tax for Order
ID: ' || p_order_id || ' ---');

   SELECT subtotal
   INTO v_total_amount
   FROM ORDERDETAIL
   WHERE ORDERID = p_order_id;

   v_final_amount := ROUND(v_total_amount*(1+v_tax),2);

   RETURN v_final_amount; -- Return the calculated count

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 0;
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred in the function: ' || SQLERRM);
        RETURN -1;
END;
/
```

Anonymous Block for Function 1:

```
SET SERVEROUTPUT ON;
SET VERIFY OFF;
ACCEPT ORDERID PROMPT 'Enter Order ID: '
DECLARE
v_order_id    orderdetail.orderid%type   := &orderid;
v_taxed_amount NUMBER(10,2);

BEGIN
v_taxed_amount := get_final_total_by_order(p_order_id => v_order_id);

dbms_output.put_line('Order ID: '|| v_order_id );
dbms_output.put_line('Total Amount: RM'|| v_taxed_amount );

END;
/
```

Example: Find final amount for Order ID = 103.

```
Function GET_FINAL_TOTAL_BY_ORDER compiled

--- Function: Getting Final Total Including Tax for Order ID: 103 ---
Order ID: 103
Total Amount: RM13.52


PL/SQL procedure successfully completed.
```

Function 2: Finding number of orders that a staff has served by staff id.

```
CREATE OR REPLACE FUNCTION get_order_served_by_staff (p_staff_id IN INT)
RETURN NUMBER
IS
    v_order_count NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('--- Function: Getting Order Count from Staff ID ' ||
p_staff_id || ' ---');

    SELECT COUNT(*)
    INTO v_order_count
    FROM "Order"
    WHERE STAFFID = p_staff_id;
    RETURN v_order_count; -- Return the calculated count
```

```
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('STAFF ID ' || p_staff_id || ' not found in ORDER
assignments.');
      RETURN 0; -- Return 0 if no order found (COUNT(*) already does this)
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('An error occurred in the function: ' || SQLERRM);
      RETURN -1; -- Return a negative value to indicate an error
END;
/
```

Anonymous Block for Function 2:

```
SET SERVEROUTPUT ON;
SET VERIFY OFF;
ACCEPT STAFFID PROMPT 'Enter Staff ID: '
DECLARE
v_staff_id    staff.staffid%type   := &staffid;
v_order_served NUMBER(10,2);
v_staff_name staff.staffname%type;
BEGIN
v_order_served := get_order_served_by_staff(p_staff_id => v_staff_id);
select staffname
into v_staff_name
from staff
where staffid = v_staff_id;
dbms_output.put_line('Staff ID: '|| v_staff_id );
dbms_output.put_line('Staff Name: '|| v_staff_name );
dbms_output.put_line('Total Order Served: '|| v_order_served);

END;
/
```

Example: Checking order done by Staff ID = 4.

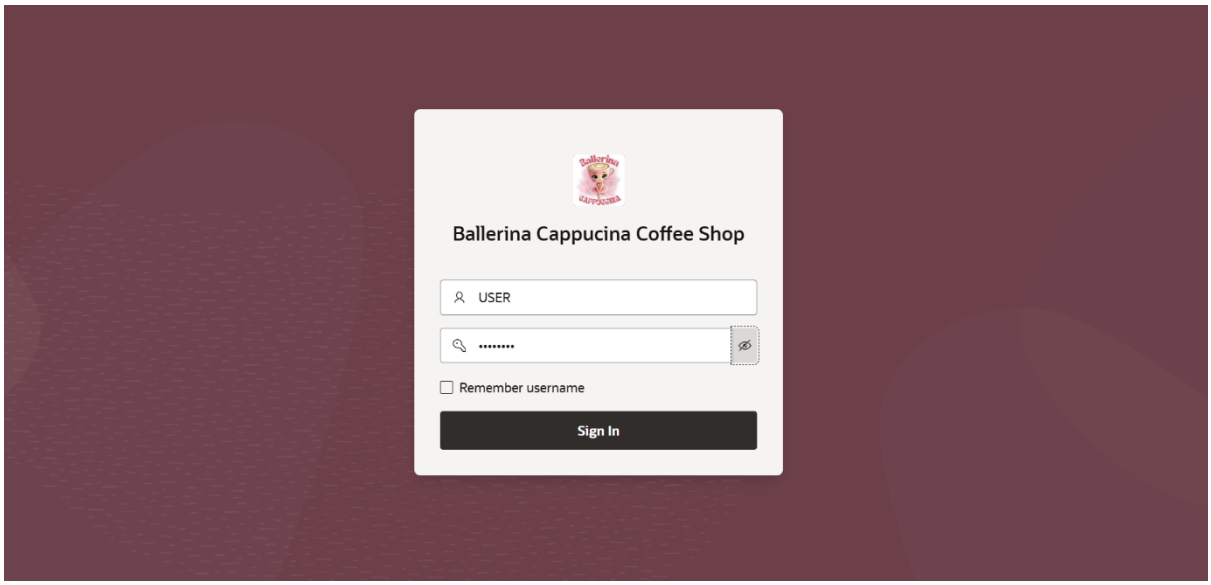
```
Function GET_ORDER_SERVED_BY_STAFF compiled

--- Function: Getting Order Count from Staff ID 4 ---
Staff ID: 4
Staff Name: Angel Batista
Total Order Served: 3

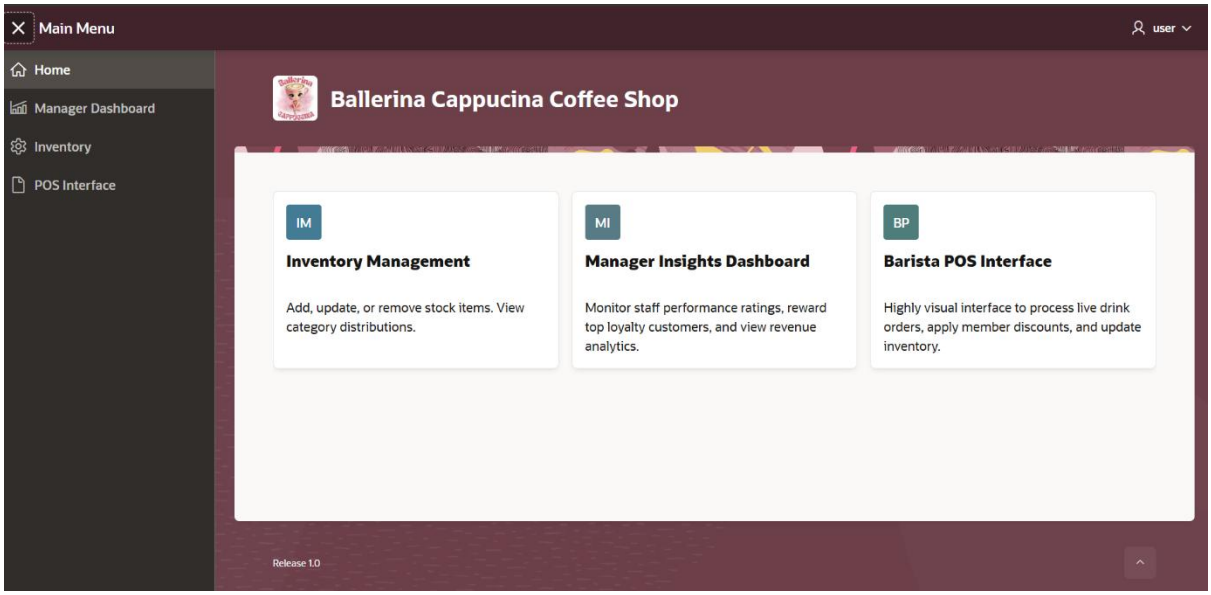

PL/SQL procedure successfully completed.
```

TASK 9

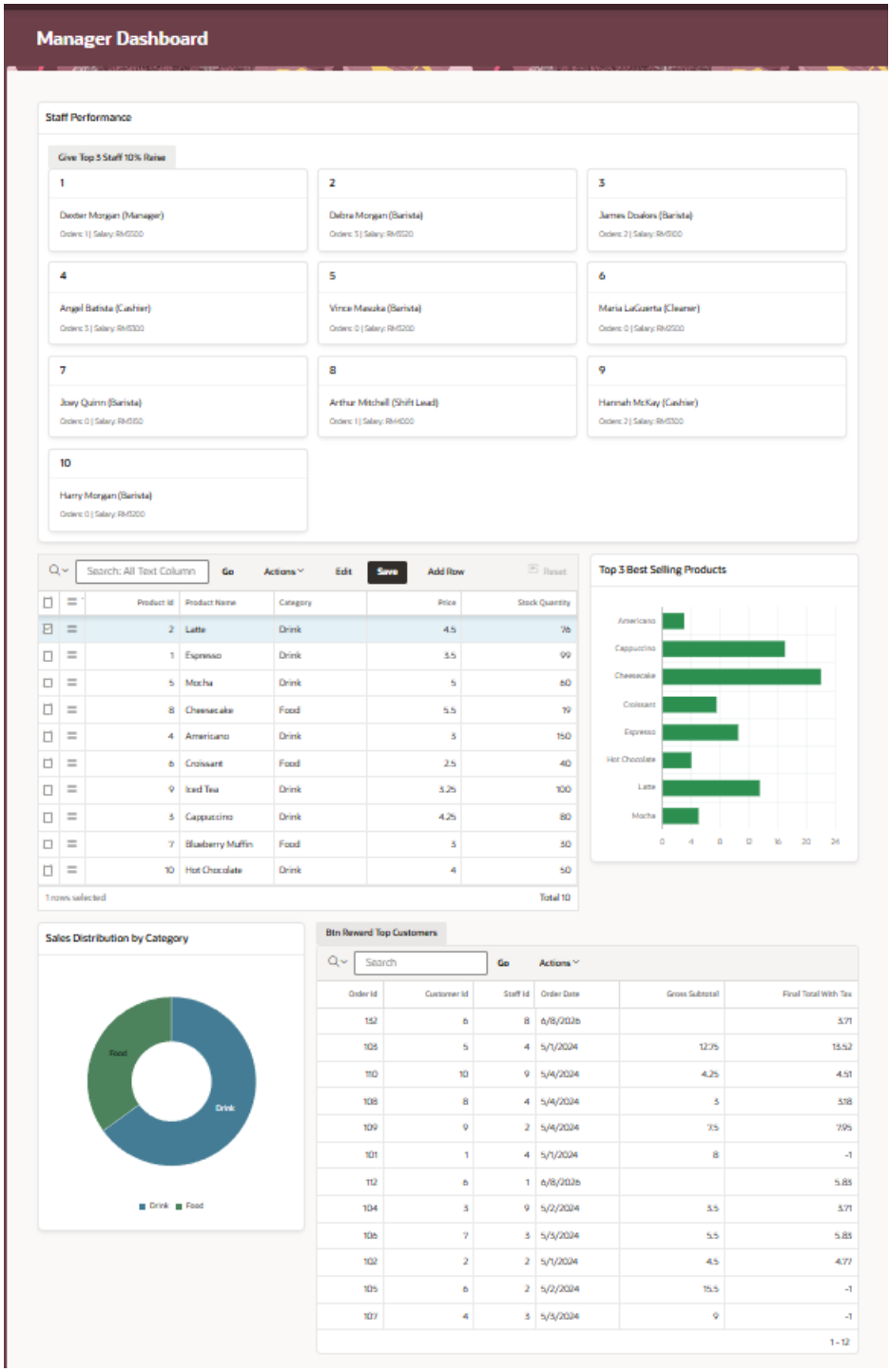
Login Page:



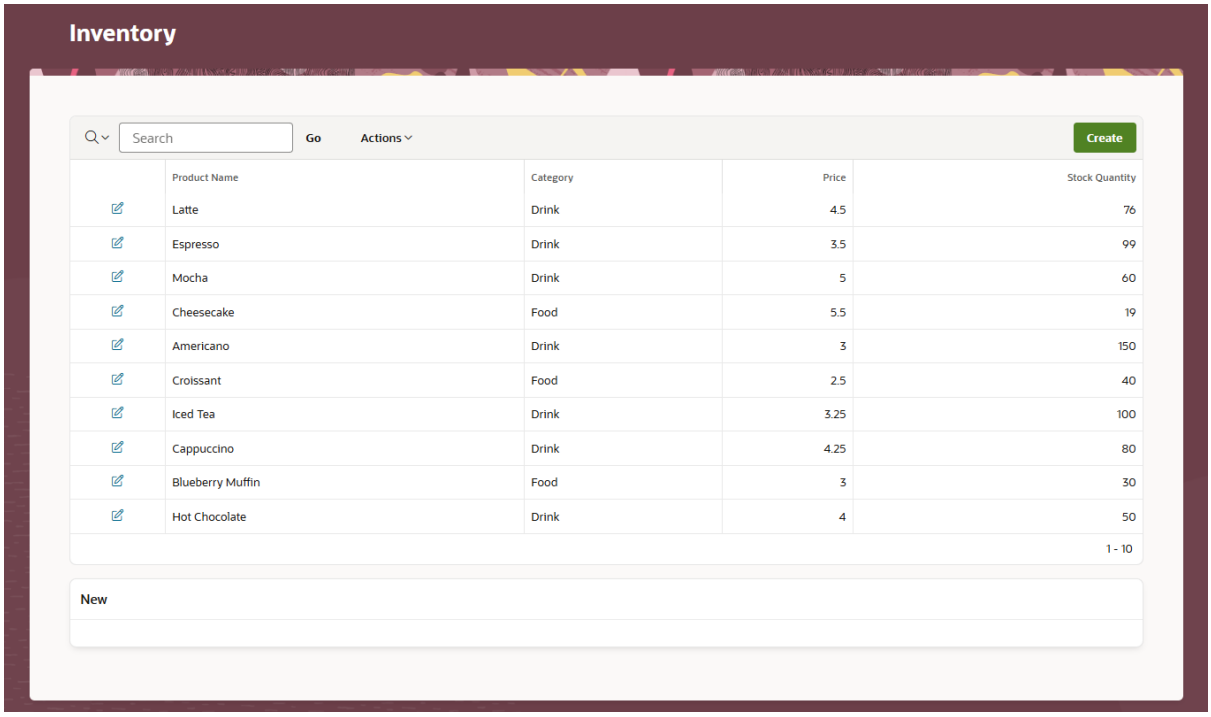
Home Page:



Manager Dashboard:



Inventory Page:



POS Interface:

